

Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО

ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

(Университет ИТМО)

Факультет инфокоммуникационных технологий

Образовательная программа Интеллектуальные системы в гуманитарной сфере

Направление подготовки 45.03.04 Интеллектуальные системы в гуманитарной сфере

О Т Ч Е Т

по курсовой работе

**Тема задания: Разработка Android-приложения с использованием
MVVM, REST API, Room, Material Design и фоновой синхронизации**

Обучающийся Дмитриева Лина Анатольевна, группы К3442

Проверяющий: Шатинский Григорий Сергеевич

Санкт-Петербург, 2024

Содержание

ВВЕДЕНИЕ.....	3
ЦЕЛЬ И ЗАДАЧИ РАБОТЫ.....	3
1. АНАЛИЗ И ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ.....	4
1.1 Общая структура приложения.....	4
1.2 Навигация.....	4
1.3 Жизненный цикл компонентов.....	5
2. УПРАВЛЕНИЕ СОСТОЯНИЕМ И MVVM.....	5
2.1 Выбор архитектуры.....	5
2.2 ViewModel и LiveData.....	6
2.3 Сохранность данных.....	6
3. РАБОТА С ДАННЫМИ И ОФЛАЙН-РЕЖИМ.....	7
3.1 Сетевое взаимодействие.....	7
3.2 Локальное хранилище (Room).....	7
3.3 Repository.....	7
4. ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС.....	8
4.1 RecyclerView и Adapter.....	8
4.2 Material Design.....	8
4.3 Пользовательские действия.....	8
5. ФОНОВАЯ СИНХРОНИЗАЦИЯ И УВЕДОМЛЕНИЯ.....	9
5.1 WorkManager.....	9
5.2 Уведомления.....	10
ЗАКЛЮЧЕНИЕ.....	11

ВВЕДЕНИЕ

Мобильные приложения являются неотъемлемой частью современной цифровой среды. Одним из наиболее востребованных типов мобильных приложений являются мессенджеры и приложения для работы с лентами сообщений, обеспечивающие обмен информацией, хранение данных и взаимодействие с пользователем в реальном времени.

Целью данной работы является поэтапная разработка Android-приложения «Мессенджер» с использованием современных архитектурных подходов, средств навигации, реактивного управления состоянием, сетевого взаимодействия, локального хранения данных и фоновой синхронизации.

В ходе выполнения работы приложение последовательно расширялось от базовой навигационной структуры до полноценного интерфейса с поддержкой офлайн-режима, пользовательских действий и фоновых уведомлений.

ЦЕЛЬ И ЗАДАЧИ РАБОТЫ

Цель работы - разработать Android-приложение «Мессенджер», соответствующее современным требованиям мобильной разработки и архитектурным принципам MVVM.

Задачи работы:

- реализовать навигацию между экранами приложения;
- обеспечить хранение и управление состоянием UI;
- реализовать загрузку данных из сети и сохранение в локальную базу;

- добавить пользовательский интерфейс с элементами Material Design;
- реализовать фоновую синхронизацию и уведомления;
- обеспечить корректную работу приложения при смене конфигурации и отсутствии сети.

1. АНАЛИЗ И ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ

1.1 Общая структура приложения

Приложение построено на основе одной главной активности, которая выступает в качестве точки входа. Основное содержимое активности реализовано через `Fragment`, что соответствует современным рекомендациям Android-разработки.

Навигация между экранами осуществляется с использованием **Bottom Navigation**, каждая вкладка которой открывает отдельный экран:

- экран ленты сообщений;
- экран профиля пользователя;
- экран настроек приложения.

Экран ленты установлен как главная страница(рисунок 1).

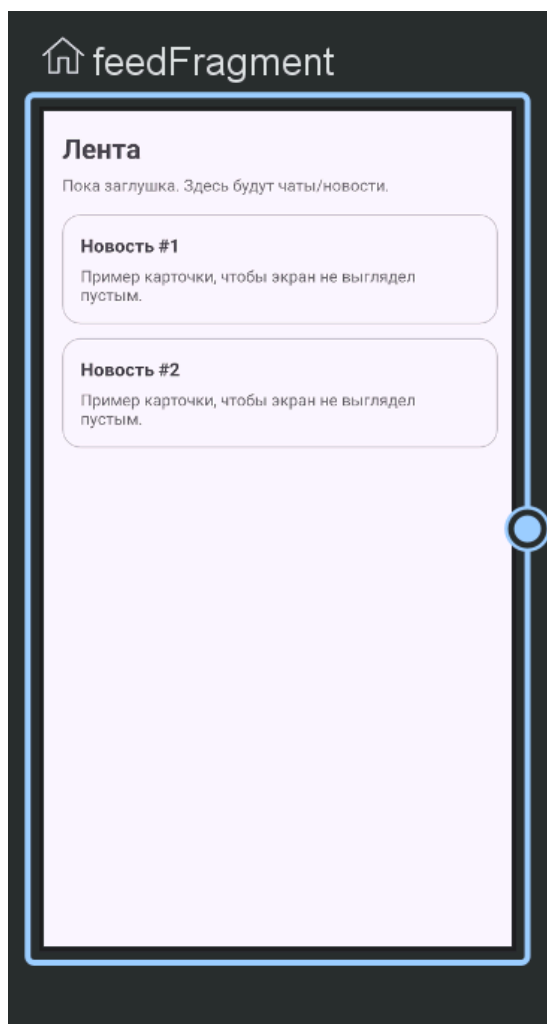


Рисунок 1 - Установка домашней страницы

1.2 Навигация

Для навигации используется Android Navigation Component:

- NavHostFragment
- NavController
- Navigation Graph

Данный подход обеспечивает безопасную навигацию и упрощённую поддержку жизненного цикла фрагментов.

1.3 Жизненный цикл компонентов

Во всех ключевых компонентах (Activity, Fragment, ViewModel) реализовано логирование этапов жизненного цикла, что позволяет отслеживать:

- создание компонентов;
- уничтожение;
- пересоздание при смене конфигурации.

2. УПРАВЛЕНИЕ СОСТОЯНИЕМ И MVVM

2.1 Выбор архитектуры

Для управления состоянием приложения выбрана архитектура MVVM (Model–View–ViewModel), обеспечивающая:

- разделение ответственности;
- независимость UI от бизнес-логики;
- устойчивость к изменениям конфигурации.

Итоговая структура на этом этапе(рисунок 2) отражает разделение ответственности между компонентами

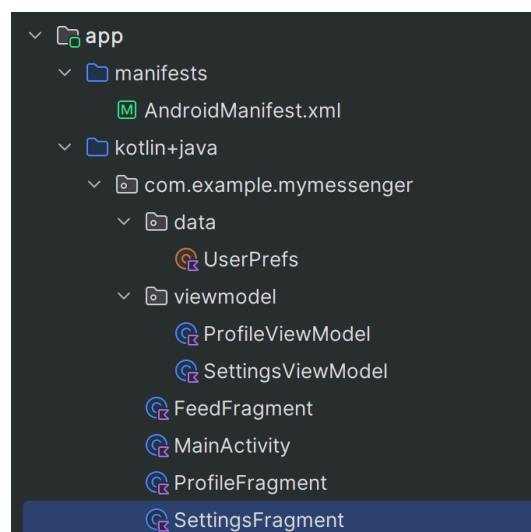


Рисунок 2 - Структура проекта

2.2 ViewModel и LiveData

Для экранов **Профиль** и **Настройки** реализованы ViewModel, хранящие в UserPrefs:

- имя и статус пользователя;
- выбранную тему оформления.

Состояние UI хранится в LiveData, что обеспечивает:

- реактивное обновление интерфейса;
- автоматическое сохранение данных при повороте экрана;
- отсутствие утечек памяти.

2.3 Сохранность данных

При изменении конфигурации (например, поворот экрана) данные не теряются, так как:

- состояние хранится во ViewModel;
- Fragment подписывается на LiveData.

3. РАБОТА С ДАННЫМИ И ОФЛАЙН-РЕЖИМ

3.1 Сетевое взаимодействие

Используется публичный REST API, возвращающий список сообщений.

Все сетевые операции выполняются асинхронно.

3.2 Локальное хранилище (Room)

Для поддержки офлайн-режима реализована база данных на основе

Room:

- Entity - модель сообщения;
- DAO - интерфейс для работы с БД;
- Database - точка доступа к базе.

3.3 Repository

MessageRepository реализован как **единая точка взаимодействия** между ViewModel и источниками данных:

- сеть;
- локальная база.

Логика работы:

- при наличии данных в базе - загружаются локальные данные;
- при наличии сети - данные обновляются и сохраняются в БД;
- при отсутствии сети - используется офлайн-кэш.

4. ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС

4.1 RecyclerView и Adapter

Для отображения списка сообщений реализован кастомный RecyclerView.Adapter, отображающий:

- имя пользователя;
- текст сообщения;
- кнопку «лайк» в виде звезды.

Каждый элемент оформлен в виде карточки.

4.2 Material Design

В интерфейсе использованы компоненты **Material Design**:

- AppBarLayout и Toolbar;
- CardView;
- FloatingActionButton.

Это обеспечивает современный внешний вид и единый стиль приложения.

4.3 Пользовательские действия

Добавлена возможность «лайкать» сообщение:

- состояние лайка хранится в базе данных;
- иконка изменяется при нажатии и закрашивается в желтый;
- состояние сохраняется между перезапусками приложения.

Визуал «лайка» показан ниже (рисунок 3)

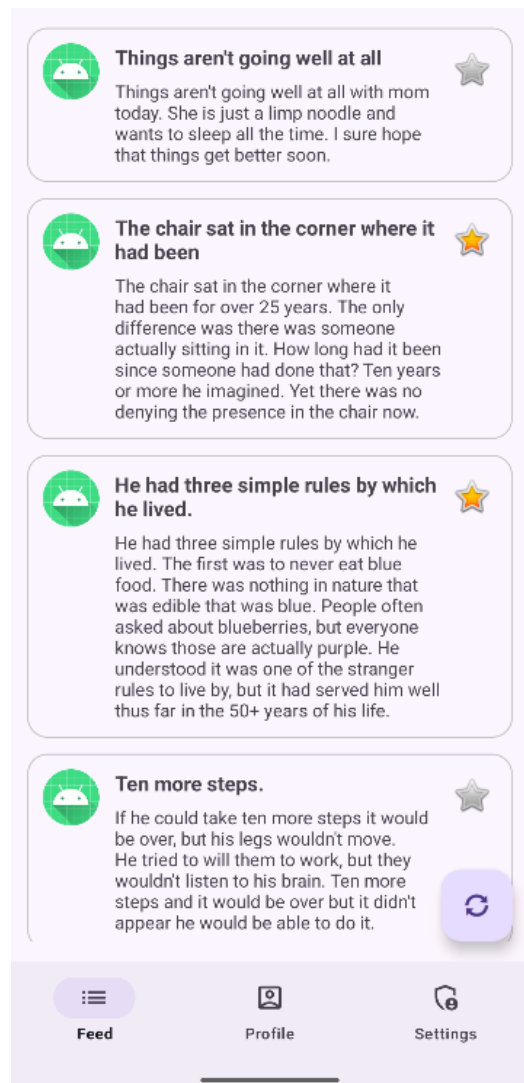


Рисунок 3 - Лайк сообщения

5. ФОНОВАЯ СИНХРОНИЗАЦИЯ И УВЕДОМЛЕНИЯ

5.1 WorkManager

Для фоновой синхронизации данных используется **WorkManager**, который:

- периодически обновляет список сообщений;
- учитывает состояние системы и ограничения Android.

5.2 Уведомления

При успешной синхронизации отображается **Notification** с текстом «Новые данные получены».

Реализовано:

- запрос разрешения на уведомления;
- корректная работа уведомлений в фоне.

Приложение отправляет уведомления, когда данные обновляются(рисунок 4).

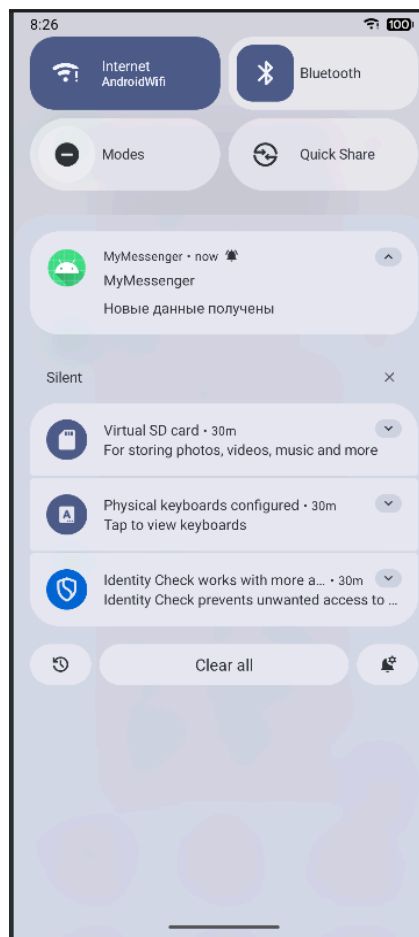


Рисунок 4 - Уведомление о получении новых данных

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы было разработано Android-приложение «Мессенджер», реализующее:

- современную навигацию;
- архитектуру MVVM;
- работу с сетью и локальной базой данных;
- офлайн-режим;
- пользовательские действия;
- фоновую синхронизацию и уведомления.

Все поставленные задачи выполнены, приложение корректно запускается и соответствует функциональным и нефункциональным требованиям.